CampusClaw

个人项目阶段性验收报告

多 AI 应用 · 提示词模板 · 知识库检索 · Skill 技能 · MCP 工具

项目	内容
项目名称	CampusClaw · 中小学教研 AI 应用平台
仓库地址	https://github.com/OhtoAi583/campusclaw
项目成员	龙浩（个人项目）
技术栈	Go（net/http）+ MySQL 8.0 + React 18 + Nginx + Docker Compose；LLM 网关 DeepSeek（OpenAI 兼容）
报告日期	2026-10-21
验收方式	平台对每次 LLM 调用做全量报文落库，报告中的参数与返回均取自真实运行记录

一、项目简介

CampusClaw 是一个面向中小学教研场景的 AI 应用平台。它把「班级知识库 + 提示词模板 + 技能 + 外部工具」组合成可配置的 AI
应用：教师选一个应用、输入问题，平台在本班材料范围内检索，按模板组装提示词、注入启用的技能、挂载 MCP 工具，再调用大模型并把出处标回原文。

与验收要求逐条对照：

验收要求	平台实现	验证入口
1. 内置多个 AI 应用	内置 4 个应用：教研问答 / 教案生成 / 材料摘要 / 学情分析，各自绑定不同模板、技能与工具	GET /api/apps；平台页应用卡片
2. 支持提示词模板	4 套模板（system + user + 变量），渲染后原样作为消息发出	GET /api/templates；平台页「提示词模板」
3. 支持知识库检索	按会话班级检索切片，按相似度取 top-k；跨班内容不进入候选集	POST /api/chat；meta.kb_chunks
4. 内置 Skill + 上传技能	3 个内置技能（引用校验 / 教研文风 / 结构化输出）；支持上传 .md 技能	GET/POST /api/skills；平台页「技能」
5. 外部工具及 MCP 接入	MCP stdio 客户端 + 仓库自带 MCP 服务器（word_count / text_stats）	GET /api/mcp/servers/{id}/tools；平台页「MCP」

二、验收方法与取证方式

为了避免「口头说明」，平台把每一次 LLM 调用完整落库：

- `llm_calls.request_json`：实际发给 `POST /chat/completions` 的完整请求体（model / messages / temperature / max_tokens / tools）；
- `llm_calls.response_json`：网关返回的原始响应（choices / usage / finish_reason）；
- `llm_calls` 同时记录 `kb_chunks`、`kb_chars`、`skill_chars`、`tool_rounds`、`round_index` 等计量字段；
- 通过 `GET /api/llm-calls/{id}` 可随时回放某次调用的完整报文，页面「本次对话发给 LLM 的原始报文」即读取该接口。

取证接口：对话与报文读取

```
POST /api/chat
{"app_id":"kb-qa","question":"导入环节为什么要用情境图？"}

GET /api/llm-calls/{id}
→ { request: {…完整请求体…}, response: {…网关原始返回…}, kb_chunks, kb_chars, skill_chars, tools }
```

下面五个问题的结论，全部引用这些真实报文；截图见第六章。

三、五个验证问题

问题 1：每次对话传给 LLM 的具体参数是什么？返回什么？提示词模板内容是否一并传递？

结论：模板内容会一并传递。 平台先渲染模板，把渲染结果原样放进 `messages[0]`（system）与 `messages[1]`（user），再连同采样参数一起发给网关。

请求字段	本次取值	说明
model	deepseek-chat	应用配置的模型
temperature	0.2	应用配置的采样温度
max_tokens	900	应用配置的输出上限
stream	False	本阶段关闭流式
messages	2 条（system 359 字、user 1133 字）	system 为渲染后的模板 + 技能；user 为模板 + 检索片段 + 问题
tools	无	未绑定 MCP 的应用不带此字段

实际请求体顶层字段：`max_tokens, messages, model, stream, temperature`

模板 `tpl-kb-qa` 渲染后的 system 消息（原文，前 220 字）：

调用 #8 · system 消息（节选）

```
你是 CampusClaw 教研助手，服务于 A 。
只依据用户消息中提供的本班材料片段作答；片段不足以回答时，直接说明「资料中未找到相关内容」。
回答中的每条事实性结论必须用 [编号] 标注出处，不得编造来源。

## 已启用技能：引用校验

# 引用校验

## 何时使用
任何基于材料的回答都必须启用。

## 规则
1. 每一条事实性结论后面必须紧跟 [编号]，编号对应用户消息里提供的材料片段序号。
2. 只能引用实际出现的片段；不得编造不存在的编号或来源。
3. 如果片段不足以支撑某个结论，直接写明「资料中未
```

模板渲染后的 user 消息（节选，含检索片段与编号）：

调用 #8 · user 消息（节选）

```
## 本班材料片段
[1] 《A班· 教研材料示例：语文阅读课教学设计》（第 2 块，字符 320-588）
## 导入环节的设计意图

导入环节用一张生活情境图引发预测，目的是把学生的注意力拉到「内容会怎么发展」上，
为后面的概括训练做铺垫。教师只提问不评价，先收集学生的不同预测，再在精读环节逐一验证。

## 分层作业设计

基础层：用一句话概括全文；提高层：找出两处关键句并说明理由；挑战层：仿写一段细节描写。

## 教学反思要点

精读时间容易被个别学生的发言拖长，需要提前约定「一句话回答」的规则；
概括能力的评价标准要提前给到学生，避免课堂上临时加码。

> 本条目属于 A 班，B 班
```

模型返回：`finish_reason=stop`，usage={"prompt_tokens": 910, "completion_tokens": 79, "total_tokens": 989, "prompt_tokens_details": {"cached_tokens": 768}, "prompt_cache_hit_tokens": 768, "prompt_cache_miss_tokens": 142}；回答前 160 字：

调用 #8 · 模型返回内容（节选）

```
导入环节用情境图，是为了引发学生预测，把注意力拉到「内容会怎么发展」上，为后面的概括训练做铺垫；教师只提问不评价，先收集
不同预测，再在精读环节逐一验证 [1]。

课堂观察也显示，使用情境图后学生预测参与率达到 90% 以上，明显高于上周的 65% [3]。
```

问题 2：绑定知识库后，每次请求是否会把知识库中的全部内容传给 LLM？

结论：不会。 每次只传「按会话班级过滤 + 相似度排序后的前 top-k 个切片」，不是整库、也不是整篇材料。

指标	数值	含义
班级知识库总切片	6	本班全部可检索切片数
本次传入切片	3	实际写进 user 消息的切片数（应用配置 top_k=4，实际命中 3 条）
传入片段字符数	1100	只占本班正文的一部分
引用条数	3	每条结果都能指回材料与字符区间

机制：检索在 SQL 层就带上 `WHERE class_id = □□□□`，再按向量相似度排序、过滤低分候选、截取 top-k。因此绑定知识库并不会把全库塞进上下文——既控制 token 成本，也避免把无关内容与跨班数据带进模型。

问题 3：绑定技能后，请求中是否包含该技能的全部内容？

结论：包含。 应用启用的技能，其 SKILL.md 正文全文会追加到 system 消息中（## □□□□□□<□□> 之后），并有总长度上限 12000 字。

指标	数值
启用技能	cite-check
注入技能字符数	237
注入位置	system 消息末尾

system 消息中技能部分的原文（节选）：

调用 #8 · system 中的技能正文（节选）

```
## 已启用技能：引用校验

# 引用校验

## 何时使用
任何基于材料的回答都必须启用。

## 规则
1. 每一条事实性结论后面必须紧跟 [编号]，编号对应用户消息里提供的材料片段序号。
2. 只能引用实际出现的片段；不得编造不存在的编号或来源。
3. 如果片段不足以支撑某个结论，直接写明「资料中未找到相关内容」，不要推测。
4. 多个片段共同支撑时写成 [1][3]。

## 输出前自检
- 逐一检查每个段落是否都有编号；
- 检查编号是否落在片段数量范围内；
- 检查是否有超出片段的推断。
```

对比：教案生成应用绑定了两个技能（引用校验 + 教研文风），system 长度从 359 字增加到 562 字，两个技能的全文都在请求里。

问题 4：执行具体技能时，其执行逻辑是什么？

结论：Skill 是「提示词层面的执行约束」，不是函数调用。 平台不执行技能里的代码，而是把技能全文注入 system，让模型按技能规定的规则生成；平台侧负责把规则落到可判定的产物上。

执行流程：

技能执行逻辑（平台侧）

```
1. 技能以 SKILL.md 形式保存（内置或上传），解析出 id / 名称 / 描述 / 正文
2. 应用通过 app_skills 绑定技能；请求时只取 enabled=1 的技能
3. 把技能正文按 `## 已启用技能：<名称>` 拼进 system 消息（总长上限 12000 字）
4. 模型依据技能规则生成（例如：每条结论必须带 [编号]）
5. 平台把生成结果与检索到的 citations 一起返回，编号与出处一一对应，可回原文核对
6. 每次调用把 system（含技能全文）、请求与返回写入 llm_calls，便于复核技能是否生效
```

以「引用校验」技能为例：它规定每条结论必须带 [□□]、编号必须在片段范围内、不得编造来源。本次回答实际输出为：

调用 #8 · 回答（带 [1][2] 编号）

```
导入环节用情境图，是为了引发学生预测，把注意力拉到「内容会怎么发展」上，为后面的概括训练做铺垫；教师只提问不评价，先收集
 不同预测，再在精读环节逐一验证 [1]。

课堂观察也显示，使用情境图后学生预测参与率达到 90% 以上，明显高于上周的 65% [3]。
```

注意区分：Skill 改变的是「模型怎么想、怎么写」；MCP 工具改变的是「模型能做什么」（见问题 5）。

问题 5：集成 MCP 工具后，每次请求传给 LLM 的参数是什么？

结论：请求多了一个 `tools` 数组（OpenAI function schema：`name` / `description` / `parameters`）。第一轮只有 system + user；模型返回 `tool_calls` 后，平台通过 MCP 执行工具，把 assistant 的 tool_calls 与 tool 结果追加进 messages，再发起第二轮请求。

轮次	messages 组成	关键字段
第 1 轮	system(352)、user(477)	tools=2 个；模型返回 2 个 tool_calls
第 2 轮	system(352)、user(477)、assistant(21)、tool(14)、tool(22)	追加 assistant(tool_calls) 与 tool(结果)，再取最终回答

`tools` 数组中的工具定义（原样来自 MCP `tools/list`）：

调用 #10 · 请求中的 tools 字段

```
- word_count：统计一段中文文本的字数与段落数
  parameters={"properties": {"text": {"description": "待统计的文本", "type": "string"}}, "required": ["text"], "type": "object"}
- text_stats：统计文本的字数、句数与平均句长，用于学情分析
  parameters={"properties": {"text": {"description": "待统计的文本", "type": "string"}}, "required": ["text"], "type": "object"}
```

第 1 轮模型返回的 tool_calls：

调用 #10 · 模型发起的工具调用

```
- word_count({"text": "# A班 课堂观察记录（第五周）\n\n## 观察对象\n五年级 A 班，共 42 人，本节为精读课文第二课时。\n\n##
  课堂结构\n导入 5 分钟，精读 …)
- text_stats({"text": "# A班 课堂观察记录（第五周）\n\n## 观察对象\n五年级 A 班，共 42 人，本节为精读课文第二课时。\n\n## 课
  堂结构\n导入 5 分钟，精读 …)
```

第二轮请求中追加的两条 tool 结果消息：

调用 #11 · 工具执行结果回填

```
tool 结果：
tool 结果：
```

最终回答（节选）：

学情分析应用 · 最终回答（节选）

```
## 一、统计结果（工具结果）

| 指标 | 数值 |
|---|---|
| 字数 | 391 字 |
| 非空段落 | 15 段 |
| 句数 | 12 句 |
| 平均句长 | 32 字/句 |

（以上均为工具结果，非材料片段内容）

## 二、学情分析

**1. 课堂参与呈“两极分化”**
导入环节使用情境图后，学生预测参与率达 90% 以上，明显高于上周的 65%，说明情境化导入对激发兴趣有效 [1]。但精读环节有 7 名学
 生连续三次发言，其余学生以倾听为主，发言分布不均 [1]。结合本班 42 人
```

四、关键设计说明

设计点	选择	理由
模板与技能放在哪里	渲染后作为 system / user 消息发送	可被完整记录与复核；换成摘要式拼接会让验收无法定位
知识库传多少	只传 top-k 命中切片	控制 token；避免无关内容干扰；跨班数据不进入候选集
技能如何执行	提示词约束 + 平台侧校验	技能不需要可执行代码，易于上传与审阅；代价是依赖模型遵循指令
MCP 如何接入	stdio + JSON-RPC，工具 schema 原样转换	不引入第三方 SDK；工具来源可审计
报文如何留存	每次调用写 llm_calls	验收与排障都需要原始请求/响应，而非事后描述
密钥管理	只存在服务端 .env，不入库不下发	浏览器永远看不到模型密钥

五、截图证据

以下截图来自真实运行（Nginx → Go → MySQL / MCP / DeepSeek），完整目录见 □□/。

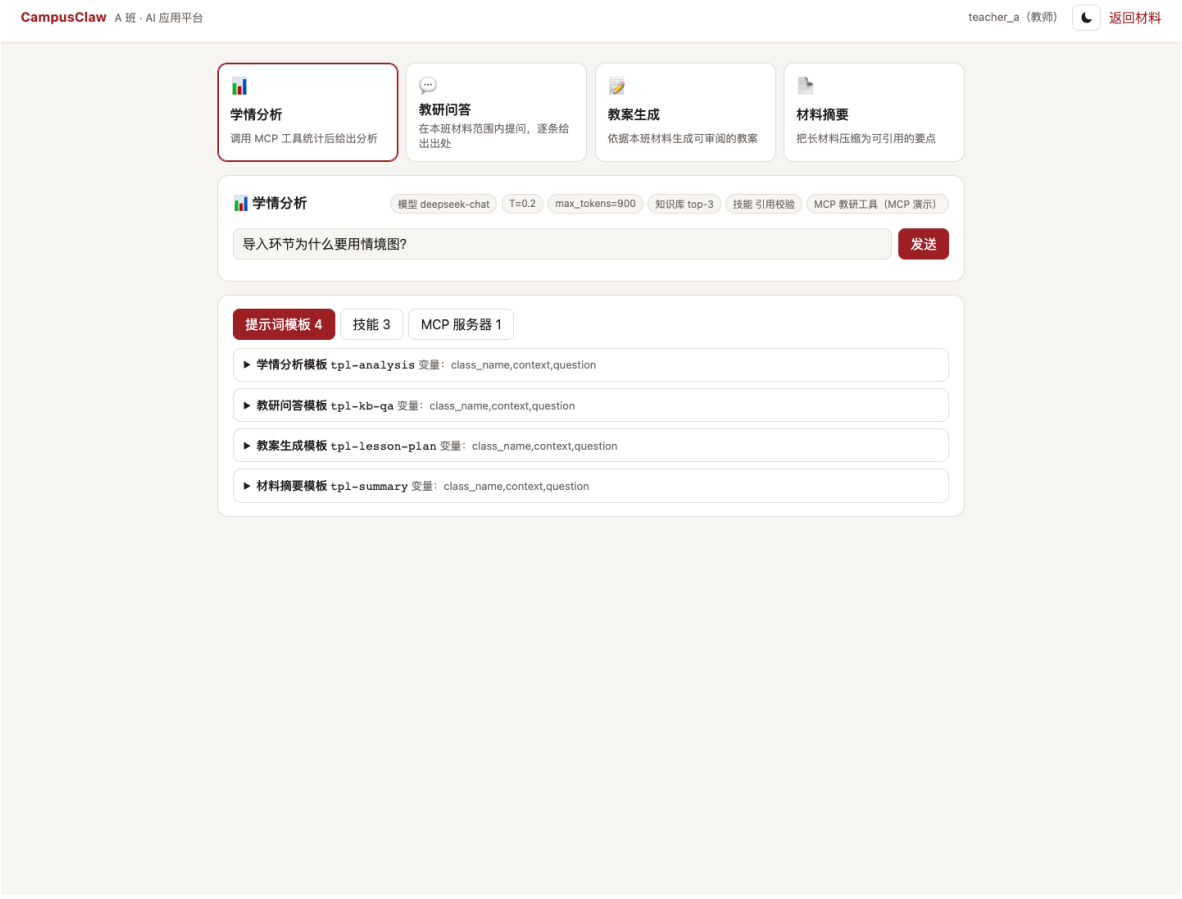


图 1 内置 4 个 AI 应用，各自绑定不同模板、技能与 MCP 工具



图 2 本次对话发给 LLM 的完整请求体与网关返回（页面直接读取 llm_calls）



图 3 提示词模板：system / user 模板与变量



图 4 技能列表：3 个内置技能 + 上传入口（.md）

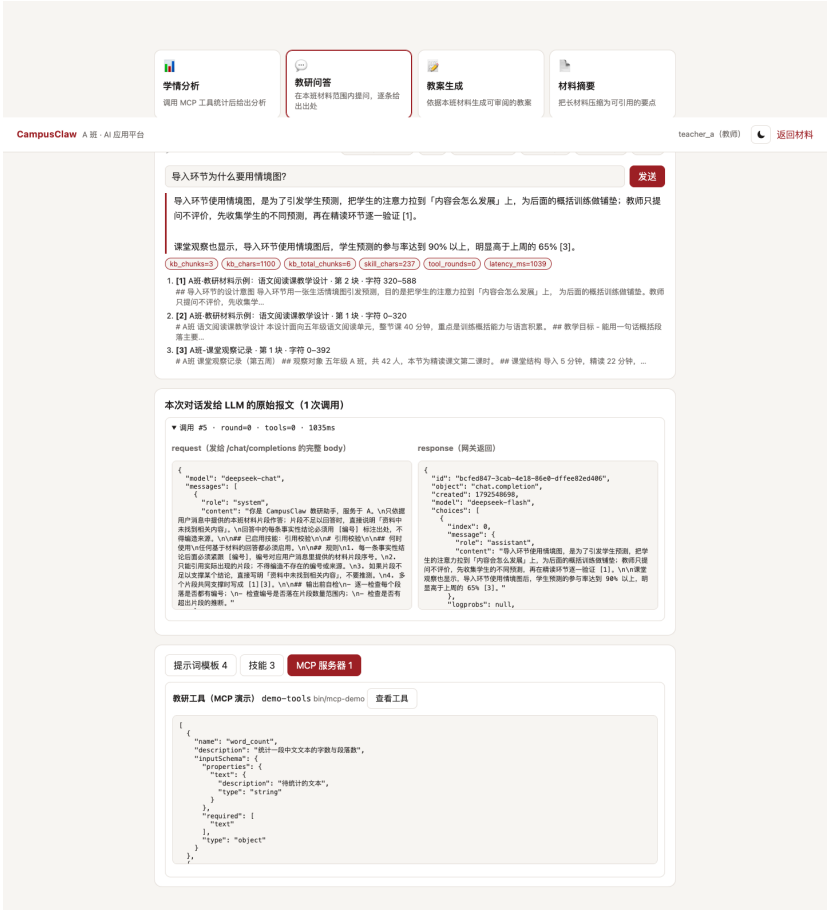


图 5 MCP 服务器与 tools/list 返回的工具 schema



图 6 MCP 工具调用：第一轮 tools + tool_calls，第二轮回填结果

六、复现步骤

从零复现

```
cp .env.example .env     # 填 DB / SESSION_SECRET / JWT_SECRET / CHAT_API_KEY
docker compose up --build -d
# 打开 http://localhost:8080 → 登录 teacher_a → 顶部「AI 应用」

# 命令行复现（本报告数据即由此产生）
python3 homework/1028-个人项目阶段性验收/collect_evidence.py
python3 homework/1028-个人项目阶段性验收/build_report.py
```

七、小结

平台已具备验收要求的五项能力，并且把「到底给模型发了什么」变成了可回放的事实：模板会随请求一并发送、知识库只传 top-k 命中切片而不是全库、技能正文会完整注入 system 消息、技能以提示词约束的方式执行、MCP 工具以 function schema 出现在请求的 tools 字段并通过两轮消息完成调用。